Building a Merchant Risk Investigation Agent with Google's ADK: From Single-Agent to Multi-Agent Systems

Why I Built This

Merchant onboarding is a high-stakes process in payments and e-commerce. Risk teams investigate whether a business should be allowed onto a platform by checking sanctions lists, adverse media, website legitimacy, and other signals.

Traditionally, analysts manually gather this evidence across multiple systems. I wanted to explore whether an AI agent could assist in this workflow.

The result was a Merchant Risk Investigation Agent built using Google's Agent Development Kit (ADK).

This project is not a production compliance system. Instead, it is an educational implementation designed to understand how modern agent frameworks work.

Understanding ADK Through a Real Use Case

I found that ADK becomes much easier to understand when viewed through the lens of a business problem.

The merchant risk investigation process naturally maps to the core concepts of agentic systems.

User Request:

"Investigate Merchant X"

↓

Agent

↓

Tools

↓

Evidence Collection

↓

Risk Assessment

↓

Structured Verdict

ADK Component 1: The Agent

The Agent is the central decision-maker.

In this project, the agent behaves like a merchant risk analyst.

```
root_agent = Agent(
    name="merchant_risk_agent",
    model=MODEL,
    instruction=INSTRUCTION,
    tools=[
        fetch_website,
        check_sanctions,
        whois_lookup,
        search_adverse_media,
    ],
)
```

The agent itself contains very little logic.

Instead, it relies on:

- a model,
- instructions,
- tools.

ADK orchestrates the reasoning loop automatically.

ADK Component 2: The Model

The model performs the reasoning.

```
MODEL = os.environ.get(
    "MODEL",
    "gemini-2.5-flash",
)
```

The model determines:

- what information is needed,
- which tools should be called,
- when enough evidence has been gathered,
- how to synthesize the findings.

Without tools, the model is simply a chatbot.

With tools, it becomes an investigator.

ADK Component 3: Instructions

I discovered that instructions are arguably the most important part of agent engineering.

```
INSTRUCTION = """
Always start by fetching the website.

Screen sanctions.

Check domain age.

Check adverse media.
"""
```

The instructions define:

- the agent's role,
- the investigation process,
- the scoring framework,
- escalation policies.

Changing the instructions significantly changes agent behavior.

ADK Component 4: Tools

Tools connect the LLM to the external world.

In this project, I implemented four tools.

Website Tool

```
fetch_website()
```

Purpose:

Determine what the merchant actually sells.

Sanctions Tool

```
check_sanctions()
```

Purpose:

Screen the merchant against sanctions lists.

WHOIS Tool

```
whois_lookup()
```

Purpose:

Determine domain age.

Recently registered domains are common fraud indicators.

Adverse Media Tool

```
search_adverse_media()
```

Purpose:

Search for fraud allegations, lawsuits, or regulatory actions.

The Most Important ADK Insight

Tools are simply Python functions.

The complexity lies not in writing the tools.

The complexity lies in teaching the agent when and how to use them effectively.

Structured Outputs

Free-form text is difficult to operationalize.

Risk systems require consistency.

I therefore defined a RiskReport schema.

The output included:

- merchant name,
- risk score,
- recommendation,
- supporting evidence.

This transformed the agent from a conversational assistant into a system capable of producing machine-readable decisions.

Moving Beyond Single Agents

The single-agent implementation worked well.

However, I encountered an important limitation.

I wanted:

- tool usage,
- structured outputs.

ADK restricts combining tools with output schemas within the same agent.

This motivated the move toward multi-agent design.

The Multi-Agent Architecture

I split the investigation process into two specialized agents.

Evidence Gatherer

↓

Risk Scorer

Evidence Gatherer

Responsibilities:

- fetch website content,
- perform sanctions screening,
- conduct WHOIS lookups,
- investigate adverse media.

Importantly:

it does not make decisions.

It only gathers facts.

Risk Scorer

Responsibilities:

- interpret evidence,
- assign risk scores,
- generate recommendations,
- produce validated RiskReports.

Importantly:

it does not use tools.

It only reasons over existing evidence.

Sequential Coordination

ADK's SequentialAgent orchestrates the workflow.

```
root_agent = SequentialAgent(
    sub_agents=[
        gatherer,
        scorer,
    ],
)
```

This design closely resembles real-world organizations.

Junior investigators collect evidence.

Senior analysts make decisions.

Lessons Learned

Building this project taught me several things.

1. Prompt engineering matters more than expected.

Small instruction changes produce dramatically different behaviors.

1. Agent systems are mostly orchestration problems.

The LLM is only one component.

1. Multi-agent systems naturally emerge from business workflows.

Specialization improves maintainability.

1. Structured outputs are critical.

Production systems require predictable interfaces.

Future Improvements

Several enhancements remain possible.

- graph-based fraud detection,

- transaction analysis tools,
- policy retrieval using RAG,
- human-in-the-loop approval workflows,
- evaluation pipelines using golden datasets.

Final Thoughts

I initially approached ADK expecting another abstraction layer around LLM APIs.

Instead, I found a framework that encourages thinking in terms of systems:

Who gathers evidence?

Who makes decisions?

How do specialists collaborate?

How should uncertainty be handled?

For me, the Merchant Risk Investigation Agent became less about building a chatbot and more about exploring how AI agents might augment complex business processes.